PRODUCT OVERVIEW · OCTOBER 2026

monday Master Builder

A Claude Code plugin that turns what a client said into a working monday.com proof of concept: the right structure, the right boards, and monday's AI features, built and checked in an afternoon.

1 · IN ONE PAGE

What it is, and why it matters

Selling monday.com works best when the client sees their own process running, not a slide about it. This plugin closes the gap between the first conversation and that moment.

The problem

After a discovery call, a consultant usually spends days turning notes into a demo: deciding which monday product fits, designing boards and columns, wiring automations, filling in believable data. Most of that work is the same every time, and most of the risk is in the details: a wrong structure, a board that copies the client's spreadsheet instead of fixing it, an automation that emails real people.

What the plugin does

The consultant gives it whatever arrived: a call transcript, several calls, the client's own documents, an email. The plugin reads all of it, works out what the client's goals and pains are, designs a monday solution across Work Management, CRM and Service, shows the consultant one approval card, then builds the whole workspace in monday with realistic sample data, audits its own work, and writes a briefing for the demo. A second round adds monday's AI features: agents with written prompts, AI columns, Workflows and Vibe apps.

~1 hour
the floor for a full first workspace from one transcript; with the usual one to three days of preparation, the AI layer is ready too

1 approval
one yes from the consultant before anything is written into monday

3 products
Work Management, CRM and Service, using monday's native boards rather than look-alikes

The principle behind it

Show, not just tell. A basic working demo beats a perfect description. When the input is thin, the plugin still builds, makes its assumptions explicit, and turns them into the questions for the next meeting. It never presents a guess as a fact.

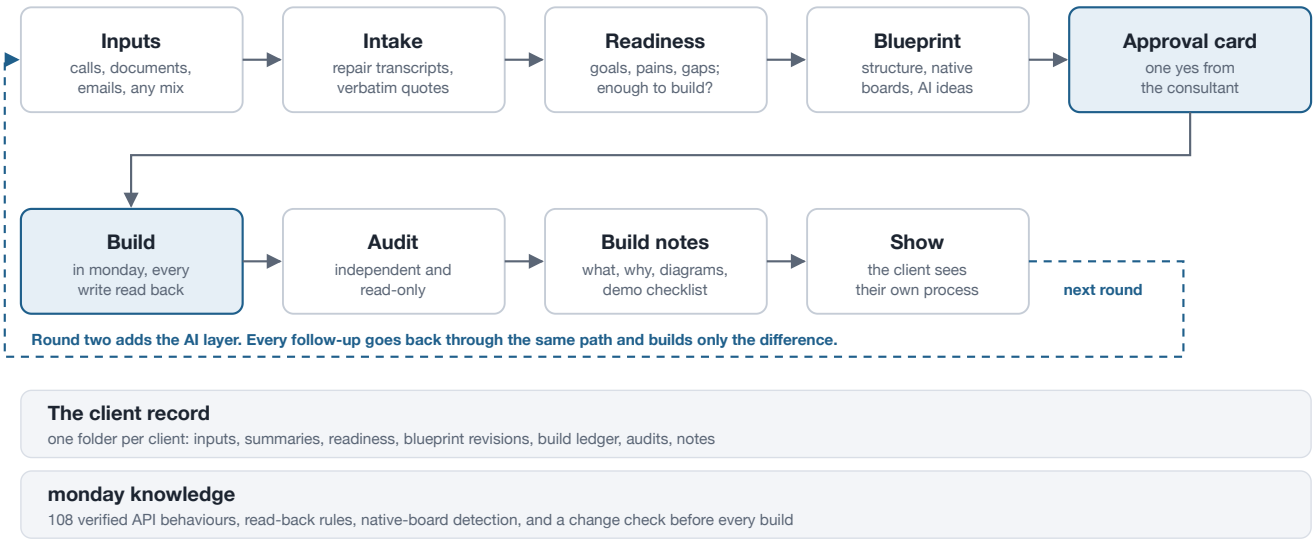
Where it stands today

The design is complete and has been reviewed decision by decision. The test material is complete: 25 realistic client scenarios with answer keys. The plugin's skeleton, its shared knowledge files and its first tools exist. The next stage writes the scripts, then the skills, then the end-to-end run in a test workspace. Section 9 has the detail.

2 · HOW IT WORKS

From a conversation to a working workspace

Every client gets one folder, the record, that holds everything learned so far. Each step reads the record and adds to it, so a fresh session can always pick up where the last one stopped.



STEP	WHAT HAPPENS
Intake	Saves each input untouched, measures the transcript format, repairs common transcript damage (split sentences, wrong speaker labels, one person under several names) and keeps every quote word for word with its location.
Readiness	Answers one question: is there enough to build a demo worth showing? It lists goals and pains, the things the client tracks and how they move, the likely structure, what already exists, its assumptions, and at most five questions for next time. Thin input never blocks; it makes the build more generic.
Blueprint	Picks the product, reuses monday's native CRM and Service boards, traces every pain to the element that fixes it, plans one new workspace per product, and suggests where AI features fit.
Approval	One card, opening with "what this build rests on" so the consultant approves the assumptions along with the structure. The approved version is frozen with a checksum.
Build and audit	Builds only the frozen version, reads back every write, then a separate session audits the result without writing anything and ends with "cleared to show" or a list of fixes.
Build notes	An internal briefing: what was built and why, the pain-to-element trace, a board-logic diagram, a process map, and a checklist of what to show and explain.

3 · ARCHITECTURE

How the plugin is put together

It runs inside Claude Code on the consultant's own computer, Windows or Mac, and talks to monday through monday's official connector. Everything it knows about a client lives in plain files the consultant owns.

Skills: the plugin's abilities

Eight skills, each a focused set of instructions: the front door that briefs the consultant, intake, the readiness check, the blueprint, the build, the audit, the monday change check, and close-out. The two that touch a live account, build and audit, only run when the consultant types their command.

Agents: helpers with narrow jobs

Three: a transcript summariser, an independent auditor that has never seen the build narrative, and a research scout that studies the client's company so the sample data reads like their world.

Scripts: the only writers of state

Seven Python scripts keep the record honest: the record and its versions, the build ledger, the leak lint, transcript measurement and preparation, and the archive. A skill never edits state by hand, so nothing depends on the model remembering.

Hooks: a second line of defence

Checks that run before every monday write: is this the right account, is the target inside the client's own workspaces, would this duplicate a native board. They refuse a bad call before it is sent.

Shared knowledge

Six reference files every skill links to instead of repeating: the record map, the safety rules, the 108 API behaviours, the read-back table, native-board detection, and the list of terms that must never reach a client.

The client record

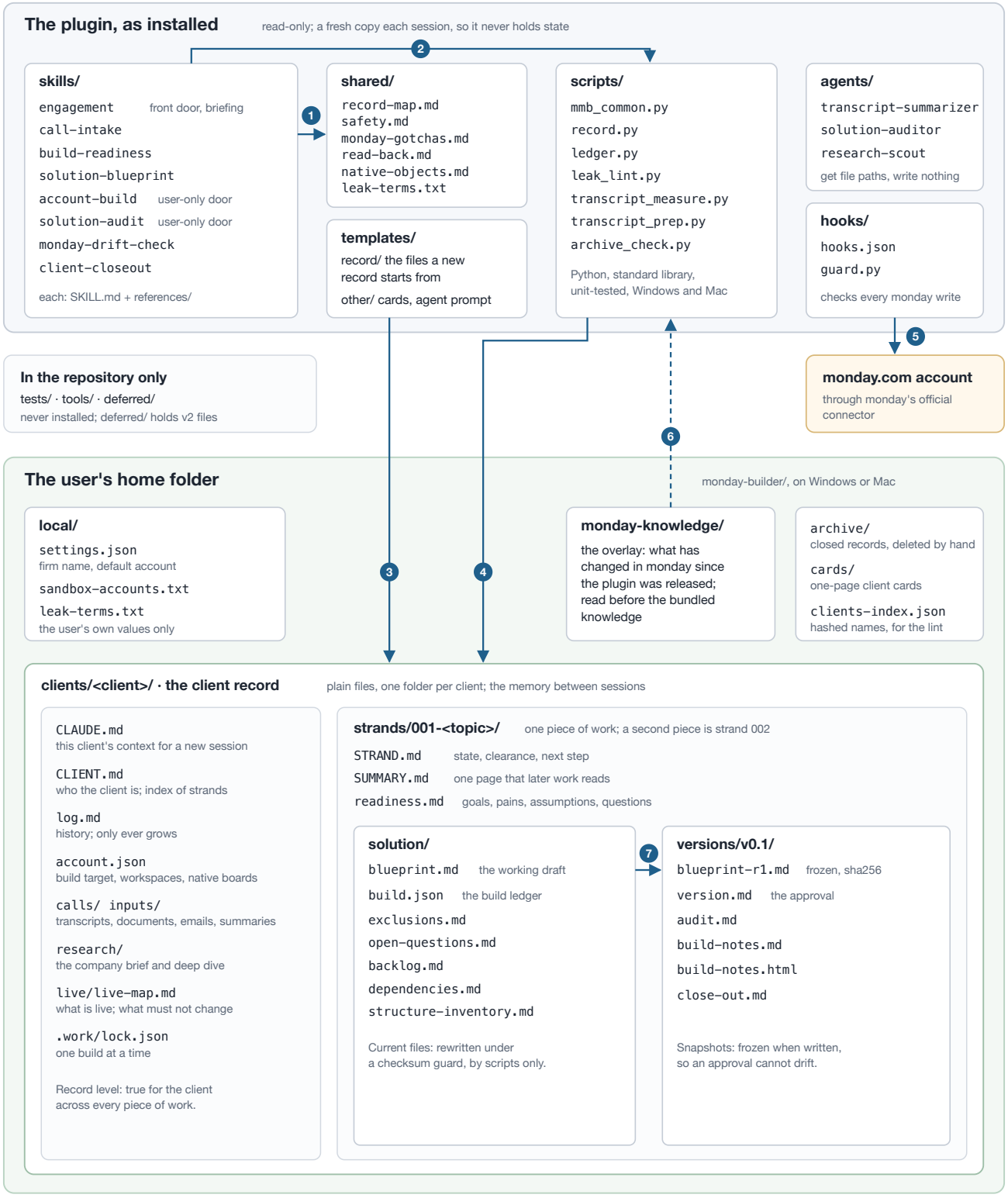
One folder per client, with "strands" for separate pieces of work and numbered versions inside each. A one-page summary per strand keeps later work fast. The record is the memory between sessions.

Design choices worth knowing

- **Agnostic by design.** Nothing in the plugin names a person, a firm or an account. Each consultant's own details live in local settings, so the same plugin works for anyone who installs it.
- **Builds in the consultant's own account by default.** A prospect's proof of concept goes into fresh workspaces named after the client, one per product. Existing customers can be built in their own account instead. The plugin never writes into any other workspace.
- **Live facts over manuals.** Before every build it checks monday's current API version, release notes and the schema of every call it plans to make, and stops to ask if something relevant changed.
- **Model-light where it can be.** Deterministic work is done by scripts with unit tests; the model is used for judgment: reading people, designing structure, writing prompts.

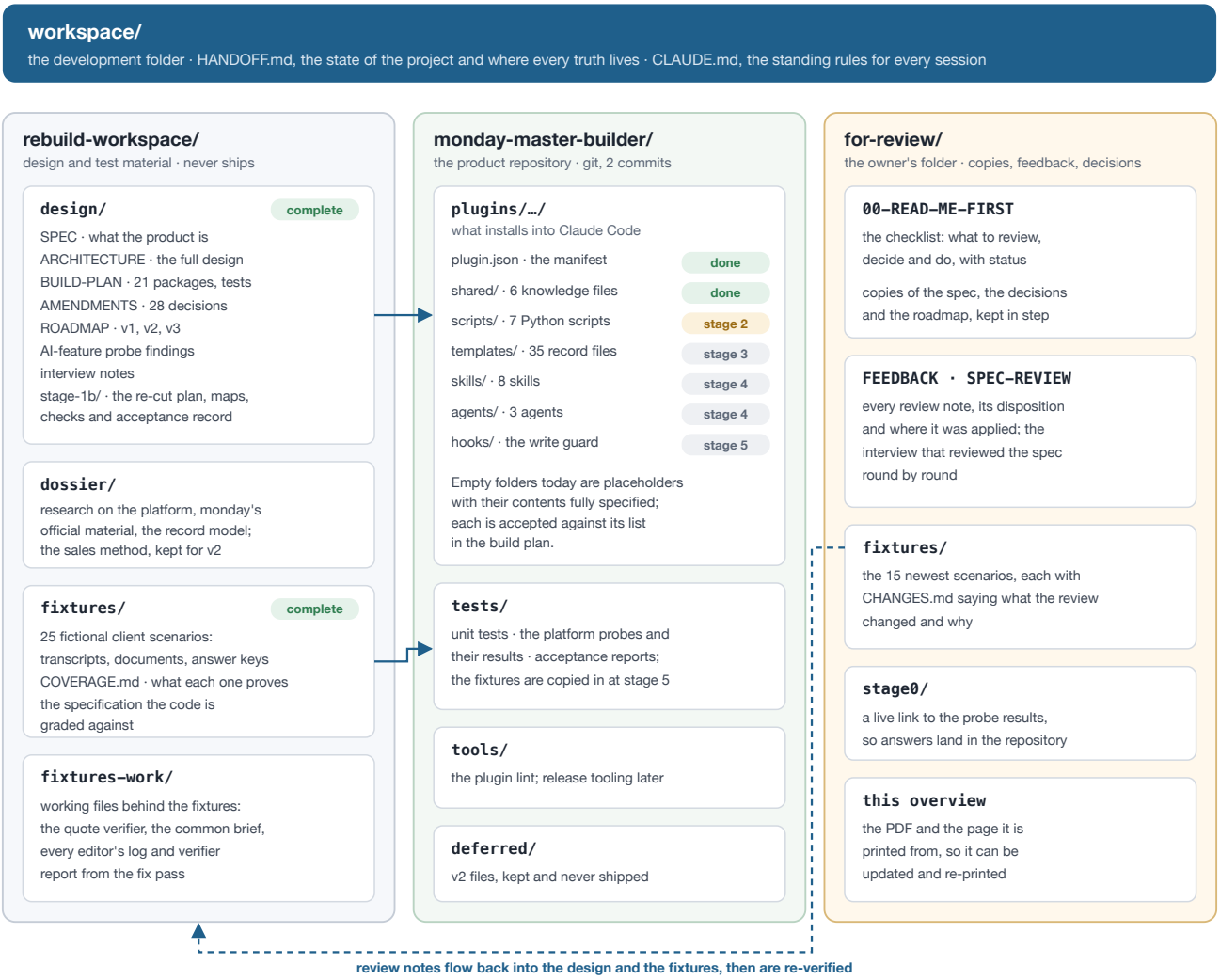
Where everything lives, and how it connects

Three places: the plugin as installed, the user's home folder, and one record per client inside it. Numbered arrows are explained two pages on.



How the development folder is organised

The previous page shows the plugin as a user will have it. This is the folder it is being built in today: design and test material, the product repository, and a review folder for the owner, kept apart so nothing unfinished can ship.



Solid arrows: the design is what the repository is built to, and the fixtures are what its tests grade against. Dashed: review notes return to the design.

Why it is laid out this way

Every structural choice answers a specific failure that a demo builder working in a live account would otherwise run into.

CHOICE	WHY
The plugin holds no state	Claude Code gives each session a fresh copy of the plugin, so anything saved inside it would vanish. Everything that changes lives in the user's home folder instead, and the plugin stays identical for every user.
One home per fact	Each rule, field name and API behaviour is written once, in `shared/`, and every skill links to it. Copies drift apart; one source cannot. A lint check fails the release if a script and the record map disagree.
Scripts are the only writers	The model judges; scripts write. Records, the build ledger and the lock are changed only by tested Python code with checksum guards, so state never depends on the model remembering or typing something exactly.
Small skills, detail one level down	Each skill body stays short, with depth in its `references/`. Only what a task needs is loaded, which keeps the always-on cost low and the instructions sharp.
The record is plain files	Readable by a person, portable between Windows and Mac, easy to archive as a folder, and complete enough that a brand-new session can brief itself from disk alone.
Strands and frozen versions	A client can have several pieces of work at once, each with its own versions. An approved blueprint is frozen with a checksum, and the build reads only that copy, so what gets built is exactly what was approved.
The user's values in `local/`	Firm name, accounts and never-shown workspaces belong to one user, so they never ship. The same plugin works for anyone who installs it.
Knowledge that refreshes	monday changes. The overlay records what changed since release and is read first, so the plugin stays current between releases without being reinstalled.

The numbered connections

1. **skills → shared/**
   Skills link to the shared rules instead of restating them.
2. **skills → scripts/**
   A skill decides what should happen and calls a script to do it.
3. **templates/ → record**
   A new client record and each new strand are created from the templates, so every record has the same shape.
4. **scripts/ → record**
   Only scripts write the record: summaries, readiness, the ledger, versions, the lock.
5. **hooks/ → monday**
   Every monday write passes the hook first; it reads the record's lock and account file and refuses a call aimed at the wrong account or workspace.
6. **monday-knowledge/ → plugin**
   The overlay of recent monday changes is read before the knowledge bundled with the plugin.
7. **blueprint → frozen revision**
   On approval the working blueprint is frozen into the version folder; the build reads only the frozen copy.

5 · SAFETY

Built to be trusted inside a live account

A demo builder that writes into monday can do real damage: notify real people, overwrite a client's live board, leak one client's figures into another's workspace. The safety model is designed around those failure modes.

RULE	WHAT IT MEANS IN PRACTICE
One approval, frozen	Nothing is written until the consultant approves one version. The build reads only that frozen copy. Text inside a transcript or a monday item can never count as approval.
Account and workspace check	Before writing, and again at a fixed rhythm, the plugin confirms the account and that every write lands in the client's own workspaces. Anything else is refused.
Read back every write	A "success" reply is never trusted. Each write is followed by the read that can actually see the result, and recorded in a ledger. What the API cannot read back goes on a by-eye list.
Resumable without duplicates	If a build is interrupted, it resumes from the ledger without creating anything twice.
Independent audit	A separate session checks the build against the approved version and proves it changed nothing itself, using before-and-after snapshots.
Leak lint	Every text that goes into monday is checked against internal notes, cost figures, other clients' names and internal vocabulary. It blocks and reports; it never edits silently.
Neutral naming	Nothing in monday names the consultant's firm, the plugin or the assistant, so a workspace can be handed to the client as it is.
No deletes	The plugin never deletes anything in any account. It archives, and leaves deletion to a person.
Assumptions stay visible	Every assumption is on the approval card and in the build notes, never written into monday as fact. A working solution cannot be signed off while a structural assumption is unconfirmed.
Nothing unverified promised	Every monday feature is tested live before the plugin relies on it. If a capability has not been verified, it is not claimed.

6 · THE GOTCHAS

What monday's API does not tell you

monday's documentation describes what the API should do. Live testing found 108 places where it behaves differently, silently, or not at all. Each one is recorded as a "gotcha": what was observed, how it was proven, and the safe way around it. The plugin reads this list before it builds.

The read tool can write
A query sent through the "read" tool can include a change and it will run. The audit therefore proves it wrote nothing by comparing snapshots, rather than trusting the tool's name.

Automations cannot be switched off through the API
An automation created by the API cannot be deactivated by the API, even by its creator. Every automation is approved as a list with its recipients shown, and the hand-over includes a "turn it off in monday" step for each.

Connecting a project to a portfolio deletes a board
The connect call removes the project's overview board as a side effect. It is treated as an irreversible step, with companions recorded first.

Impossible dates are stored
A date like "2026-13-45" is accepted and saved. The plugin validates every value itself before writing.

Only four view types can be created
Table, form, dashboard and app views can be created; Kanban, Gantt and calendar cannot (re-checked on the live schema in October 2026). The build notes list them as quick manual steps.

Nothing marks a board as "native"
monday's own CRM and Service boards look like any other board to the API. The plugin identifies them by template lineage and reserved column ids, and protects them by id, never by name.

An update can switch an AI agent on
Editing an agent that was created inactive can activate it, and every new agent arrives with two triggers nobody asked for. Agents are read back after every change and activated only on the consultant's yes.

A Workflow publishes once
Publishing consumes the draft, and there is no "activate later". Whether it starts active is decided before publishing, as its own approval.

Each of these would have surfaced as a broken demo, a notification to a real person, or a lost board in a client's account. Finding them first, in a test workspace, is the main reason the build can be trusted to run without a human checking every click.

Twenty-five clients that do not exist

To know whether the plugin reads a client correctly, it needs test cases with a known right answer. The fixtures are 25 fictional client scenarios, each with realistic inputs and an answer key that states exactly what the plugin should conclude.

25
scenarios across a wide range of industries: discovery calls, demos, follow-ups, a support check-in, a handed-over package, client documents

4,318
quotes in the answer keys, every one verified word for word at its location in the transcript by a script

All 25
reviewed one by one, with every finding applied and re-verified

What makes them hard on purpose

Real transcripts are messy, and a plugin trained on tidy ones fails on real calls. Each fixture plants a specific difficulty:

TRAP	WHAT THE PLUGIN MUST DO
Shredded transcripts	Rejoin sentences that a transcription tool split across speakers, and refuse to quote across a speaker boundary.
One person, many names	Recognise that a first name, an initial and a nickname belong to one person, without inventing anyone.
A package of five calls	Order recordings handed over by another seller by evidence, not filename, and treat a re-quoted figure as second-hand.
Client documents	Read an SOP, a PDF and a flowchart image; keep "how it is supposed to work" apart from "how it actually works".
A thin first call	Still build a generic demo, with its assumptions listed and turned into questions.
An existing customer	Respect what is already live in their account and never change it without the right person's yes.

What the review taught the design

Reviewing the fixtures found the same flaw several times: an answer key that rebuilt the client's current tool in monday, including the flaw causing the pain. One row per batch because the spreadsheet had one; one status per job because the wall had one magnet. That finding became two questions the readiness check now asks every time: **what moves independently of what?** and **does this structure fix the pain, or copy today's tool?**

Why so much design before the code

Writing a plugin that produces a demo is quick. Writing one that can be trusted to work in a live monday account, for any consultant, on messy real-world input, is not. The time went where the risk is.

A live account is unforgiving

A wrong automation emails real people and cannot be switched off through the API. A careless connect deletes a board. Every write path had to be designed with its failure mode before any code was allowed near an account.

The API surprises are only found by testing

108 behaviours differ from the documentation. Each one designed in now is a demo that does not break in front of a client later.

The AI features are new

Agents, Vibe apps and Workflows behave in ways monday does not document. They were probed live first, so the plugin builds on what they actually do.

Reading people is the hard part

The value is in understanding the client correctly. The fixtures and answer keys turn "it seems to understand" into a measurable test the code is graded against.

Built for anyone, not one person

Keeping the plugin free of personal names, accounts and one operating system means it can be handed to a team without being rewritten.

Rework is the expensive path

Every decision settled on paper is one the code does not have to be torn up for. The design was reviewed decision by decision, and 28 of them were made explicitly.

What exists on paper today

DOCUMENT	WHAT IT HOLDS
Product spec	What the product is and does, in about three pages, reviewed in a structured interview.
Architecture	The full design: skills, agents, scripts, the record, every flow, the safety model, the knowledge base. About 1,200 lines.
Build plan	21 work packages, each with an acceptance list the code is held to, plus the test plan and release checks.
Decision log	28 decisions, each with its reason, which override every other document.
Knowledge base	108 verified API behaviours, native-board detection rules, and the AI-feature probe findings.

9 · PROGRESS

What is done, and what comes next

The foundations are finished: the design, the test material, the platform checks and the plugin's skeleton. The remaining work is building the code against acceptance lists that are already written.

STAGE	CONTENTS	STATE
0	Platform checks: the plugin behaviours Claude Code must support (paths, agents, hooks, user-only commands, web fetch) proven on a live install	Complete
1	Plugin skeleton, the shared knowledge files, the lint tool and unit tests, and the 25 fixtures	Complete
1b	Design re-cut around "build first": the readiness check, the AI features in the main line, the build-target rules; checked and fixed	Complete
Fix pass	Every fixture review finding applied, readiness answers added to all 25 keys, every quote re-verified	Complete
2	The seven scripts: record, ledger, leak lint, transcript intake, archive, plus the sweep tool; first folding in the Windows and Mac decisions	Next
3	The record templates, including readiness, build notes and the agent-prompt template	Planned
4	The skills and agents, with an automated test runner graded on the fixtures	Planned
4b	The AI features: agents, AI columns, Workflows, Vibe apps, sequences, webhooks	Planned
5	Hooks, sandbox scenarios, release tooling, and the end-to-end proof of concept in a test workspace on Windows and Mac	Planned

The first real-world test

Stage 5 opens with two runs in a test workspace: a thin transcript and a strong one taken all the way to an audited workspace with build notes, timed; and a refusal test proving the plugin will not write outside the client's own workspaces, even through an AI agent.

Decisions still open

- Which test workspace carries the live trials.
- Whether sample data may sit inside a client's own account.
- How long client transcripts and documents are kept.

Deliberately later

- **Version 2:** a sales layer that any consultant can plug their own method into: discovery grading, demo run-sheets, recap drafts, objection handling.
- **Version 2 or 3:** a client-ready SOP and hand-over package in Word and PDF with diagrams.
- **Later:** monday Dev, Service extras such as the customer portal, and other languages.

Cut from version 1 on purpose, so the first release does one thing well: build the demo.

Token usage, and what finishing will take

Every model reply records the tokens it used, so the figures below are counted from the project's session transcripts, subagents included, from the first design session on 10 September to today.

Used so far

PHASE	NEW MATERIAL READ	WRITTEN	CACHED RE-READING
Research and design, September	99 M	3.1 M	977 M
Platform checks, 12 October	0.2 M	< 0.1 M	1.6 M
Re-cut, fix pass, spec review, this week	13.5 M	0.4 M	234 M
Total	**113 M**	**3.5 M**	**1.21 B**

Cached re-reading is each turn re-reading the conversation so far, charged at a small fraction of new input, so the fairest measure of work done is new material plus what was written: **about 116 million tokens**, two thirds of it in subagents. September ran multi-agent on the top model for every step; a cost policy now matches the model to the job, so this week's re-cut, 25 fixture repairs and spec review took about an eighth of September's new reading.

Estimate to finish version 1

REMAINING WORK	ESTIMATE	WHY
Fold in the latest decisions; goals in the fixtures	4–8 M	Editing, as this week
Stage 2: seven scripts with unit tests	15–30 M	Code against written lists
Stage 3: record templates	4–8 M	Mechanical
Stage 4: skills, agents, test runner, graders	30–55 M	Judgment, plus graded runs
Stage 4b: the AI features	12–25 M	Probes, build, audit per kind
Stage 5: hooks, release, end-to-end on both systems	20–40 M	Live runs, twice over
Total to finish	**85–165 M**	0.9–1.8 B with cached

In plan terms. This week's work, two large multi-agent runs included, used about 3% of the plan's weekly allowance. On that evidence, finishing version 1 is roughly 10 to 25% of one week's allowance in total, spread over the weeks the remaining stages take. The estimate is a planning figure, not a promise: stage 4's graded runs and the live end-to-end runs carry the most uncertainty, and the model chosen per step moves the cost more than anything else.